DIGISANCHIKA DOCUMENT API AND TOKEN REFERENCE

CURRENT API BASE URL

The Flutter app currently uses this default base URL:

https://digisanchika.acstechnologies.co.in/api

The URL can be overridden by BASE_URL or API_BASE_URL in .env. The current .env file does not define either variabl

The Dart code calls paths such as /documents; with the default base URL, the final URL is:

https://digisanchika.acstechnologies.co.in/api/documents

HOW THE APP STORES AND SENDS TOKENS

After login, the app reads these response fields:

? Access token: ‘accessToken‘, ‘access_token‘, or ‘token‘
? Refresh token: ‘refreshToken‘, ‘refresh_token‘, or ‘refresh‘

Tokens are stored securely using Flutter Secure Storage:

? ‘access_token‘
? ‘refresh_token‘

Every request made through ApiClient.instance.dio automatically adds:

Authorization: Bearer <access_token>

When the API returns 401, the app calls POST /api/auth/refresh with the refresh token, saves the returned tokens, a

TOKEN COMPARISON BY REQUEST TYPE

API request	Token behavior	Reason
Login	No existing access token	Creates the session token
Normal document/folder API	Bearer access token automatically added	Uses ApiClient.instance.dio

Publish/unpublish	Bearer access token automatically added	Protected user action
Download URL request	Bearer access token automatically added	Server checks permission
Download from presigned URL	Usually no Bearer token	URL already contains temporary authorization
Upload URL request	Bearer access token automatically added	Server creates upload permission
PUT to presigned upload URL	Usually no Bearer token	Storage service validates the signed URL
Token refresh	Refresh token in request body	Refresh endpoint issues a new access token

AUTHENTICATION APIS

POST /api/auth/login

Request fields accepted by the app:

json
{
"email": "employee-or-email",
"employeeId": "employee-or-email",
"employee_id": "employee-or-email",
"username": "employee-or-email",
"password": "password"
}

The response must return an access token using one of accessToken, access_token, or token. It should also return a

POST /api/auth/refresh

json
{
"refreshToken": "refresh-token",
"refresh_token": "refresh-token"
}

GET /api/auth/me

Returns the authenticated user used by the Home, Documents, Library, sharing, and permission screens.

DOCUMENT APIS USED BY THE CURRENT UI

Method	URL	Token	Purpose
GET	/api/documents	Bearer	Load Home and My Documents files
GET	/api/documents?folderId={folderId}	Bearer	Load files inside a folder
GET	/api/documents/{documentId}	Bearer	Load document details
DELETE	/api/documents/{documentId}	Bearer	Delete a document
GET	/api/documents/{documentId}/view-url	Bearer	Get a preview/view URL
GET	/api/documents/{documentId}/download-url	Bearer	Check permission and get download URL
POST	/api/documents/{documentId}/publish	Bearer	Publish a document to Library
POST	/api/documents/{documentId}/unpublish	Bearer	Remove a document from Library

The document response should include id, name, original_filename, folder_id, mime_type, file_size_bytes, allow_down

FOLDER APIS USED BY THE CURRENT UI

Method	URL	Token	Purpose
GET	/api/folders	Bearer	Load folder cards
POST	/api/folders	Bearer	Create a folder
GET	/api/folders/{folderId}	Bearer	Load folder metadata and hierarchy
DELETE	/api/folders/{folderId}	Bearer	Delete a folder
POST	/api/folders/{folderId}/publish	Bearer	Publish a complete folder tree
POST	/api/folders/{folderId}/unpublish	Bearer	Unpublish a folder
GET	/api/folders/{folderId}/contents	Bearer	Browse folder contents, if supported

The folder response should preserve id, name, parent_id, created_at, documents, and child folders.

LIBRARY APIS

Method	URL	Token	Purpose
GET	/api/library	Bearer	Load published documents and folders
GET	/api/library/folders	Bearer	Load published folder cards
GET	/api/library/folders/{folderId}	Bearer	Browse a published folder

The publish response should contain the new state:

json
{
"success": true,

"folderId": "folderId",
"isPublishedToLibrary": true
}

UPLOAD APIS

Method	URL	Token	Purpose
POST	/api/documents/upload-url	Bearer	Create a signed upload URL
PUT	/api/documents/upload/{documentId}	Signed URL rules	Upload file bytes
POST	/api/documents/{documentId}/confirm-upload	Bearer	Finalize upload metadata
POST	/api/documents/bulk-upload-urls	Bearer	Create multiple signed upload URLs

Upload metadata must preserve folderId, fileName, fileSize, mimeType, and allowDownload.

DOWNLOAD APIS

Method	URL	Token	Purpose
GET	/api/folders/{folderId}/download-url	Bearer	Check folder access and get ZIP URL
GET	/api/folders/{folderId}/download	Bearer or signed URL	Download ZIP bytes
GET	/api/folders/{folderId}/download-manifest	Bearer	Return recursive folder/file manifest

The ZIP must preserve the root folder, nested folders, original file names, and permitted documents.

IMPORTANT CURRENT BACKEND CHECK

The Flutter client is prepared to use bearer tokens, but the local Backend/app/auth.py login response currently ret

Therefore, the production authentication API must return token fields recognized by the Flutter client. Otherwise,

Do not put access tokens or refresh tokens in URLs. Do not include real token values in logs or documentation.